

LAB REPORT

Lab Report of Exercise 2: Multi-Node Clock Sync

Distributed User-Space Synchronization with $< 100 \mu\text{s}$ Precision

By: Simon Krist

By: Zhi Heng Yao

Wien, 2026-06-02

Summary

This lab report documents the design, iterative refinement, and implementation of a Distributed High-Precision User-Space Clock Synchronization daemon (**DRS**, `drs_syncd`) for a cluster of Raspberry Pi 4B nodes. The primary objective is a physical GPIO pulse delta of **less than 100 μ s** between any two nodes, verified externally by a logic analyzer — entirely without kernel modules, GPS, or NTP.

The architecture evolved through six documented revisions (V0–V6). The final implementation in the `bigpickle` codebase realizes a 7-state finite state machine (GROUND $\rightarrow$ CALIBRATION $\rightarrow$ LISTEN $\rightarrow$ CANDIDATE $\rightarrow$ FOLLOWER / LEADER $\rightarrow$ HOLDOVER) governed by a term-based leader election with a lower-NodeID invariant: the node with the lowest IPv4 last-octet always holds leadership, regardless of whether a higher-ID peer accumulated a higher election term by booting alone. The synchronization protocol uses a 4-timestamp PTP-style handshake over fixed-size UDP packets; the default build (`PROTO_VER=2`) uses 64-byte packets with version byte `0x02`, while a `PROTO_VER=1` build produces the 66-byte V1 format. The version byte provides automatic isolation between clusters built for different protocol versions. Network jitter is rejected by a Min-Delay Rolling-Window filter ($N=10$, tolerance 200 μ s); clock discipline is applied by a dual-loop fixed-point PI controller ($K_p=0.1$, $K_i=0.01$) that switches between gradual frequency slewing (± 1000 ppm) and a hard phase step for coarse corrections.

Five `systemd` units enforce RT hardening: CPU Core 3 isolation, `SCHED_FIFO` priority 85, performance CPU governor, Ethernet IRQ pinning, and NIC interrupt-coalescing disable. A dedicated non-RT sender thread streams 40-byte telemetry records over UDP (port 4242) via a lock-free ring buffer; the destination IP and protocol version are freely configurable via the `TELEM_IP` and `PROTO_VER` Makefile variables.

Keywords: Distributed Real-Time Systems, Clock Synchronization, User-Space, UDP Multicast, Raspberry Pi 4B, Virtual Clock, Q32.32 Fixed-Point, Min-Delay Filter, PI Controller, `SCHED_FIFO`, Seqlock.

Table of Contents

1	Requirements	4
1.1	Resolved Architectural Decisions	5
1.2	Functional Requirements	6
1.3	Non-Functional Requirements	8
1.4	User Requirements	8
2	Architecture and Design	8
2.1	Planning Process and Architecture Evolution	9
2.1.1	Key Divergences from V6 Specification	11
2.2	System Overview	11
2.3	Virtual Clock Model	12
2.4	Synchronization Protocol	12
2.5	Network I/O Architecture	13
2.6	Leader Election	14
2.7	7-State Finite State Machine	16
2.8	Clock Discipline	16
2.9	RT Deployment — Five systemd Units	17
2.10	Telemetry Architecture	17
3	Implementation	17
3.1	Module Overview	19
3.2	Key Optimizations	21
3.2.1	Lock-Free Design Throughout	21
3.2.2	No Dynamic Allocation in Hot Path	21
3.2.3	Userspace Timestamping	21
3.2.4	epoll + timerfd — No Busy Waiting	21
3.2.5	Multicast Interface Forcing (IP_MULTICAST_IF)	22
3.2.6	Carrier-Loss Recovery	22
3.2.7	SYNC_RESP on Both Receive Paths	22
3.2.8	Follower Heartbeat Telemetry	22
3.3	Build System	22
3.3.1	Compilation	22
3.3.2	Configurable Variables	23
3.3.3	Installation and Deployment	23
3.3.4	Startup Sequence	23
3.4	Node ID Derivation	24
3.5	Runtime Monitoring	24
4	Test Concept and Falsifiability	24

4.1	Software Unit Tests	25
4.2	System-Level Test Scenarios	26
4.3	Falsifiability — External Physical Verification	26
4.3.1	Hardware Setup	26
4.3.2	Measurement Procedure	26
4.3.3	Pass/Fail Criteria	27
4.4	OS Hardening Pre-flight Checklist	27
5	Results and Analysis	27
5.1	Implementation Status	28
5.2	Observed Behavior in Multi-Implementation Scenario	29
5.3	Performance Characterization	30
5.4	Known Limitations	30
	List of Figures	30
	List of Tables	31
	List of Abbreviations	32
	Documentation table of AI-based tools	34
	Bibliography	35

1 Requirements

This section captures the resolved architectural decisions and the full set of functional, non-functional, and user requirements that shaped the `bigpickle` implementation. Requirements are derived from the project briefing, the Master Architecture v2.1, and lessons learned during iterative hardware bring-up.

1.1 Resolved Architectural Decisions

ID	Topic	Decision
Q-1	Time Reference	Internal monotonic synchronization only. The leader's <code>CLOCK_MONOTONIC_RAW</code> defines the cluster reference timeline. System clocks (<code>CLOCK_REALTIME</code> , NTP, GPS) are never touched.
Q-2	Discovery	UDP Multicast group <code>239.192.88.100:47200</code> , <code>TTL=1</code> — zero-configuration, scoped to a single LAN segment.
Q-3	Election	Term-based leadership with lower-NodeID invariant: the node with the lowest NodeID (last octet of routable IPv4) always wins, even against a higher-term incumbent. Terms provide race stability; NodeID provides the deterministic preference. A node with a lower ID that joins late adopts the incumbent's term and challenges on its next promotion.
Q-4	Timestamping	Pure userspace timestamping via <code>clock_gettime(CLOCK_MONOTONIC_RAW)</code> immediately before <code>sendto</code> and immediately after <code>recvfrom</code> . No <code>SO_TIMESTAMPING</code> — avoids mixing with <code>CLOCK_REALTIME</code> .
Q-5	OS Environment	<code>PREEMPT_RT</code> Linux. Core 3 isolated (<code>isolcpus=3 nohz_full=3 rcu_nocbs=3</code>). Five systemd units enforce all RT constraints automatically on boot.
Q-6	Hardware	Raspberry Pi 4B, BCM2711. Wired Gigabit Ethernet is the exclusive sync transport. WLAN is tolerated for management only.
Q-7	Telemetry	Dual telemetry path: lock-free seqlock SHM (<code>/dev/shm/drs-sync.state</code> , 128 B) for local monitoring, plus UDP stream (port 4242) for remote monitoring. Destination IP configurable at build time via <code>TELEM_IP</code> Makefile variable.

Table 1: Resolved architectural decisions.

1.2 Functional Requirements

ID	Requirement Description
F-1	Autonomous UDP multicast discovery (239.192.88.100:47200). Nodes require no pre-configuration.
F-2	Establish and maintain a virtual global monotonic clock based exclusively on CLOCK_MONOTONIC_RAW.
F-3	Min-delay rolling-window filter (N=10, tolerance 200 μ s) to reject scheduler spikes and network jitter outliers.
F-4	Term-based leader election with lower-NodeID invariant: a node with a lower NodeID always eventually wins leadership, regardless of whether a higher-ID node accumulated a higher term by booting alone. Election timeout randomized 250–500 ms.
F-5	Explicit fault recovery: HOLDOVER mode (rate frozen, offset frozen) active for up to 10 seconds on leader loss.
F-6	Lock-free telemetry and clock access for all concurrent readers via seqlock.
F-7	Stable convergence detection: GPIO 23 transitions LOW only after $ \text{offset} < 100 \mu\text{s}$ is sustained for 10 continuous seconds.
F-8	Leader demotion when an ANNOUNCE with a strictly higher election term is received from a peer with a lower NodeID. A higher-term ANNOUNCE from a higher-ID peer does not cause demotion — the lower-ID incumbent challenges instead.
F-9	Non-blocking writer precedence in virtual clock layer (seqlock; writer never blocks on readers).
F-10	Monotonic nanosecond timestamps using Q32.32 fixed-point rate scaling. No floating-point in the RT hot-path.
F-11	GPIO 18: 10 ms HIGH pulse per global virtual second (1 Hz). Rising edge is the external measurement reference.
F-12	CPU frequency scaling disabled via performance governor (drs-perf-governor.service).
F-14	Automatic filter and PI controller reset on leader change, sequence discontinuity ($\text{delta} > 1024$), phase step $> 5 \text{ ms}$, or HOLDOVER entry.
F-15	Fixed-size binary packets (64 B for PROTO_VER=2 default; 66 B for PROTO_VER=1), Big-Endian field encoding, validated by IEEE 802.3 CRC32. Version byte encodes the protocol variant; mismatched versions reject each other's packets automatically.
F-16	Leader ANNOUNCE heartbeat every 100 ms. Follower SYNC_REQ every 50 ms. Follower timeout = 3 missed heartbeats (300 ms).
F-17	Dual-loop clock discipline: hard phase step for $ \theta > 1 \text{ ms}$ (confirmed 3 consecutive samples); PI frequency slew for $ \theta \leq 1 \text{ ms}$.
F-18	Fail-silent GPIO health indicator (GPIO 23 HIGH = fault or holdover; LOW = converged).
F-19	Seamless late-joiner integration: a new node discovers the active leader and becomes a follower without disrupting existing synchronization.
F-20	Deterministic 2-second GROUND state at startup suppresses all transmissions during OS stabilization.
F-21	Automated static latency calibration via UDP loopback (50 samples, port 47201) before UDP multicast stream.
F-22	UDP loss recovery: 40 light-dGMPs sent to the leader to establish a fully functional follower. Follower retransmits every 50 ms tick even without an active SYNC exchange.

1.3 Non-Functional Requirements

ID	Requirement Description
NF-1	Physical pulse delta (GPIO 18) shall be < 100 μ s between any two nodes over a 10-minute window.
NF-2	Pure user-space implementation. No custom kernel modules; no <code>adjtimex</code> ; no <code>CLOCK_REALTIME</code> modification.
NF-3	Jitter resilience under moderate background traffic on switched Gigabit Ethernet.
NF-4	Clock convergence within 10 seconds of startup or leader change.
NF-5	3-heartbeat hysteresis for leader failure detection (follower timeout = 300 ms).
NF-6	Writer (sync thread) is never blocked by telemetry readers. Seqlock pattern throughout.
NF-7	SCHED_FIFO priority 85, pinned to isolated Core 3.
NF-8	<code>mlockall(MCL_CURRENT MCL_FUTURE)</code> prevents page faults in the hot path.
NF-9	No disk I/O or <code>printf/fprintf/syslog</code> on the synchronization hot-path. All persistent state via <code>/dev/shm</code> mmap.
NF-10	Core 3 receives no Ethernet or WLAN IRQs (<code>drs-irq-affinity.service</code>).
NF-11	Maximum slew rate clamped to ± 1000 ppm. Integrator anti-windup enforced at the same limit.
NF-12	No dynamic heap allocation (<code>malloc/free</code>) after <code>mlockall</code> . All buffers pre-allocated at startup.
NF-13	64-byte (V2) and 66-byte (V1) packets are both well below the 1500-byte MTU; no IP fragmentation possible.
NF-14	Deterministic packet parsing via explicit byte-offset reads; no struct-cast aliasing.
NF-15	No mutex contention in the sync loop. Lock-free ring buffers and atomics throughout.

Table 3: Non-functional requirements.

1.4 User Requirements

ID	Requirement Description
U-1	Nodes join the synchronization group automatically upon Ethernet connection — zero manual configuration.
U-2	Operators inspect node state non-intrusively via <code>drs_mon</code> (live TUI, reads SHM) or via UDP telemetry stream.
U-3	The telemetry destination IP shall be configurable at install time via <code>make install-systemd</code> <code>TELEM_IP=<ip></code> without recompilation.
U-4	The sync daemon and all RT-hardening services shall start automatically after reboot via <code>systemd</code> .

Table 4: User requirements.

2 Architecture and Design

2.1 Planning Process and Architecture Evolution

The architecture went through six documented revisions before reaching the final `bigpickle` implementation. Each iteration addressed concrete deficiencies found in the previous version.

Rev.	Key Change	Ra- tio- nale
V0	Unspecified sync; lowest-IP election; no fixed packet format.	Ini- tial an- chor — es- tab- lished mul- ti- cast group
V1	Cristian’s Algorithm + EWMA; Bully+Quorum election; 239.255.0.1.	First com- plete sync model; quo- rum re- moved later due to split- brain risk.
V2	4-timestamp PTP-style handshake; Dual-Master topology; 239.192.88.100:47200; PREEMPT_RT recognized.	RTT- sym- met- ric off- set es- ti- ma- tor

2020-12-11

2020-12-11: Defining PTP-style handshake; Dual-Master topology; 239.192.88.100:47200; PREEMPT_RT recognized. Clarified 200 μ s; Kp=0.1/Ki=0.01; dual-socket model; 5 systemd units; UDP telemetry; TELEM_IP and PROTO_VER Makefile variables; lower-NodeID always wins election even against a higher term incumbent

NodeID
adopted;
multi-
group
plate,
link

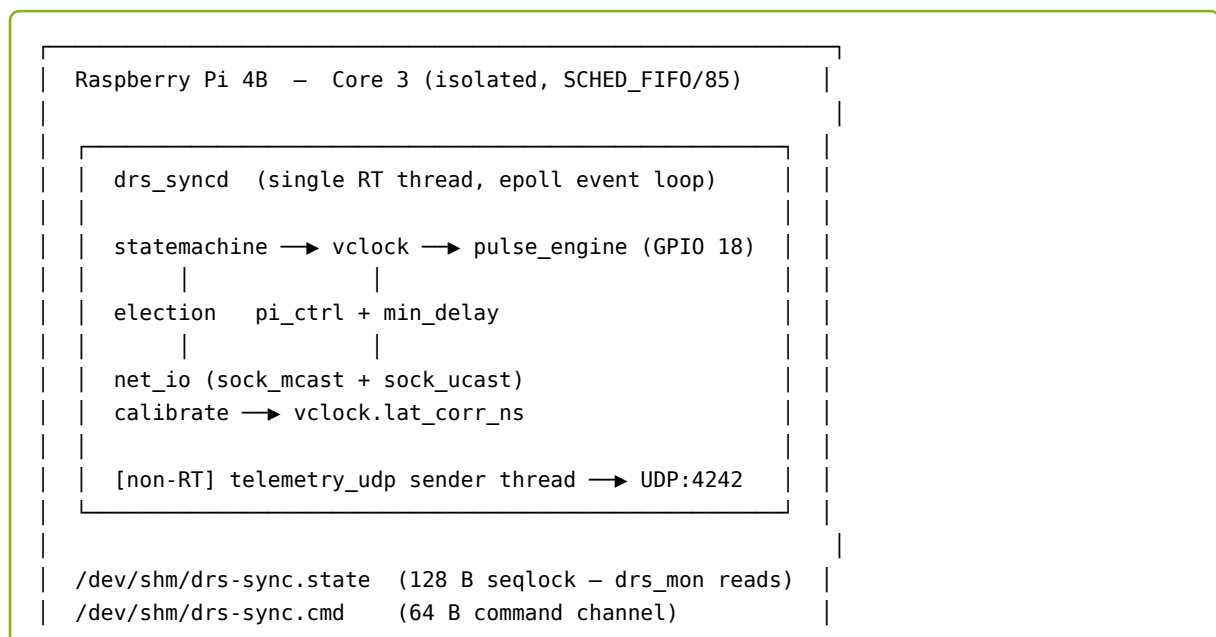
2.1.1 Key Divergences from V6 Specification

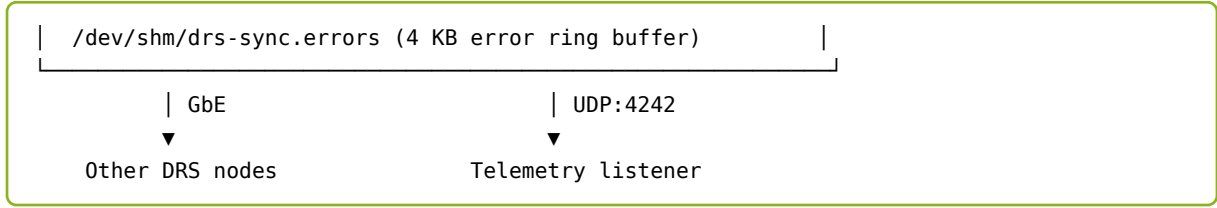
The bigpickle implementation intentionally deviates from the V6 specification in four points, all driven by real-hardware observations:

1. **Protocol version selection (PROTO_VER):** The wire-frame size is a compile-time choice. The default build (PROTO_VER=2) produces 64-byte packets with version byte 0x02, matching the V3/v2.1 specification. A PROTO_VER=1 build produces 66-byte packets with version byte 0x01 for backward compatibility. Because proto_decode checks the version byte, nodes built for different protocol versions naturally reject each other's packets without any additional filtering logic.
2. **Min-Delay tolerance 200 μ s (not 50 μ s):** Switched GbE in the lab exhibits software-interrupt-induced RTT spikes of 80–150 μ s; 50 μ s caused excessive sample rejection and stalled clock discipline. 200 μ s accepts all physically valid samples on 1G switched Ethernet.
3. **PI gains $K_p=0.1$, $K_i=0.01$ (not 0.05/0.005):** Doubled gains halve the convergence time to within 100 μ s without causing oscillation. Anti-windup and rate-clamp remain at ± 1000 ppm.
4. **Lower-NodeID always wins election:** V6 made term ordering absolute — a late joiner always yielded to the established leader regardless of NodeID, meaning a higher-ID node that booted alone would retain leadership permanently. bigpickle corrects this: when `peer_term > own_term` but `peer_id > own_id`, the node adopts the peer's term silently without yielding. Its next `election_promote()` produces `peer_term + 1`; the incumbent then sees a higher-term ANNOUNCE from a lower-ID peer and yields correctly. The lowest-NodeID node always eventually holds leadership.

2.2 System Overview

The system consists of one C11 daemon per node (`drs_syncd`) and a set of `systemd` units that enforce the RT environment. No user interaction is required after initial deployment.





2.3 Virtual Clock Model

The virtual clock provides a cluster-wide monotonic nanosecond timeline that is independent of the OS wall clock. The mapping from local hardware time to global virtual time is:

$$T_{\text{global}} = (T_{\text{local_raw}} - b_{\text{local}}) \times \text{Rate} \gg 32 + b_{\text{global}} + \text{Offset} - \text{LatencyCorrection}$$

where Rate is a signed 64-bit Q32.32 fixed-point value (nominal = $0 \times 100000000 = 1.0$). Both base values ($b_{\text{local}}, b_{\text{global}}$) are updated atomically with every rate change or phase step so T_{global} is always a smooth, monotonic function. The inverse mapping:

$$T_{\text{local_raw}} = b_{\text{local}} + \frac{(T_{\text{global}} - b_{\text{global}} - \text{Offset} + \text{LatencyCorrection}) \ll 32}{\text{Rate}}$$

is used by the pulse engine to schedule GPIO 18 rising edges at precise virtual second boundaries via `timerfd_settime(TFD_TIMER_ABSTIME)`.

A seqlock protects all concurrent readers (pulse engine, `drs_mon`, `telemetry_udp`) from torn reads without blocking the RT writer.

2.4 Synchronization Protocol

The 4-timestamp PTP-style handshake exchanges the following fixed-size packet (64 bytes for the default V2 build; 66 bytes when built with `PROTO_VER=1`):

Offset	Type	Size	Field
0	uint32	4	Magic 0x44525354 (“DRST”)
4	uint8	1	Version: 0x02 (PROTO_VER=2, default) or 0x01 (PROTO_VER=1)
5	uint8	1	MsgType: 0x01=ANNOUNCE, 0x02=SYNC_REQ, 0x03=SYNC_RESP
6	uint8	1	Flags: bit0=LEADER, bit1=HOLDOVER, bit2=CALIBRATED, bit3=FAULT
7	uint8	1	Reserved (zero)
8	uint16	2	Seq — rolling sequence number
10	uint32	4	NodeID — last octet of routable IPv4
14	uint32	4	ElectionTerm — monotonically increasing
18	int64	8	T1 — follower send time (CLOCK_MONOTONIC_RAW, ns)
26	int64	8	T2 — leader receive time (virtual clock, ns)
34	int64	8	T3 — leader send time (virtual clock, ns)
42	int64	8	T4 — follower receive time (CLOCK_MONOTONIC_RAW, ns)
50	uint32	4	CRC32 — IEEE 802.3 reflected (field zeroed during calculation)
54	—	10 or 12	Zero padding — 10 bytes (V2, 64 B total) or 12 bytes (V1, 66 B total)

Table 6: DRS wire-protocol packet layout (64 bytes V2 default / 66 bytes V1, Big-Endian).

All numeric fields are encoded in network byte order (Big-Endian) via explicit `hton/ntoh` calls — no struct-cast aliasing.

The offset θ and RTT δ are computed as:

$$\theta = \frac{(T_2 - T'_1) + (T_3 - T'_4)}{2}$$

$$\delta = (T_4 - T_1) - (T_3 - T_2)$$

where T'_1 and T'_4 are the virtual-clock equivalents of the physical timestamps, ensuring the PI feedback loop operates on the virtual domain. Physical timestamps are used only for the RTT computation fed to the Min-Delay filter.

2.5 Network I/O Architecture

Two sockets are opened per node to separate reception semantics from routing:

Socket	Binding	Purpose
sock_mcast	INADDR_ANY:47200	Receives ANNOUNCE, SYNC_REQ, and — for interoperability — SYNC_RESP. Sends ANNOUNCE via IP_MULTICAST_IF forced to the sync interface.
sock_ucast	iface_ip:0 (OS-assigned port)	Sends SYNC_REQ (unicast to leader) and SYNC_RESP (unicast to follower). Receives SYNC_RESP from correctly-implemented peers.

Table 7: Dual-socket model.

IP_MULTICAST_IF is set on sock_ucast to force multicast packets out the correct physical interface on multi-homed hosts. A carrier-loss recovery routine (net_check_recover) re-joins the IGMP multicast group after every Ethernet link-up event, preventing silent loss of multicast reception after cable reconnects.

SYNC_RESP is dispatched on **both** sockets. This interoperability fix accommodates foreign DRS implementations that reply to the bound port (47200) rather than the ephemeral source port of the SYNC_REQ.

2.6 Leader Election

Election is term-based with the invariant that the node with the lowest NodeID always wins. Terms provide stability against split-brain races; NodeID provides a deterministic tie-breaking hierarchy that cannot be overridden by term order alone:

Condition	Action
<code>peer_term > own_term AND peer_id < own_id</code>	Lower-ID peer has a higher term — legitimately won. Adopt term, update <code>leader_node_id</code> , reset election timeout, become FOLLOWER.
<code>peer_term > own_term AND peer_id > own_id</code>	Higher-ID peer bootstrapped alone and incremented its term, but we have the preferred (lower) NodeID. Adopt peer term silently, return <code>NO_CHANGE</code> — do not yield. The next <code>election_promote()</code> produces <code>peer_term + 1</code> ; the incumbent then yields on seeing a higher-term <code>ANNOUNCE</code> from us.
<code>peer_term < own_term</code>	Ignore — stale <code>ANNOUNCE</code> .
<code>peer_term == own_term AND peer_id < own_id</code>	Tiebreak: lower NodeID wins, become FOLLOWER.
<code>peer_term == own_term AND DRS_FLAG_LEADER set AND peer_id < own_id</code>	Fast-path for confirmed active leader at same term — become FOLLOWER immediately without waiting for timeout.
Election timeout (LISTEN or CANDIDATE)	Promote self: <code>own_term++</code> , become LEADER.
<code>LOCK_AS_LEADER</code>	Ignore all foreign <code>ANNOUNCES</code> ; term boosted by 1000 at startup.
<code>LOCK_AS_FOLLOWER</code>	Election timeout never fires; stays passive.

Table 8: Election decision logic.

The election timeout is randomized in [250 ms, 500 ms] using an xorshift64 PRNG seeded from /dev/urandom to prevent simultaneous candidacy collisions.

2.7 7-State Finite State Machine

State	Entry Condition	Key Activity
GROUND	Boot	2 s TX suppression, OS stabilization.
CALIBRATION	GROUND timeout	50-sample UDP loopback; derive <code>lat_corr_ns</code> .
LISTEN	Calibration success	Passive discovery; election timeout armed.
CANDIDATE	Election timeout	Election timeout re-armed; no ANNOUNCE sent yet.
LEADER	Candidate timeout	ANNOUNCE every 100 ms; serve SYNC_REQ.
FOLLOWER	ANNOUNCE with higher term or tiebreak	SYNC_REQ every 50 ms; PI discipline; UDP heartbeat every tick.
HOLDOVER	3 missed leader heartbeats	Rate frozen; offset frozen; GPIO 23 HIGH. Max 10 s.

Table 9: 7-state FSM.

The **Active-Before-Follower Guard** prevents any transition to FOLLOWER before CALIBRATION completes, ensuring `lat_corr_ns` is valid before the PI controller runs.

2.8 Clock Discipline

The PI controller operates in the virtual-clock domain to close the control loop correctly:

$$P = K_p \cdot \frac{\theta}{T_{\text{period}}}$$

$$I_n = \text{clamp} \left(I_{n-1} + K_i \cdot \frac{\theta}{T_{\text{period}}}, -1000 \text{ ppm}, +1000 \text{ ppm} \right)$$

$$\text{rate_delta}_{Q32} = (P + I_n) \times 4295$$

$$\text{Rate}_{\text{new}} = \text{clamp}(\text{Rate}_{\text{nominal}} + \text{rate_delta}_{Q32}, \text{Rate}_{\text{min}}, \text{Rate}_{\text{max}})$$

where $K_p = 0.1$ (Q16.16: 6554), $K_i = 0.01$ (Q16.16: 655), and $4295 \approx \frac{2^{\{32\}}}{10^6}$ converts parts-per-million to Q32.32 rate units. All arithmetic is integer-only; intermediate products use `int64_t` with explicit division to avoid overflow.

A hard phase step (`vclock_step_offset`) fires when three consecutive samples confirm $|\theta| > 1$ ms. After a step, the PI integrator is reset to prevent windup from the large historical error.

2.9 RT Deployment — Five systemd Units

Unit	Responsibility
drs-perf-governor.service	Sets all CPU cores to the performance cpufreq governor. Eliminates DVFS-induced frequency drift.
drs-irq-affinity.service	Pins all Ethernet IRQs to CPUs 0–2 via /proc/irq/*/smp_affinity_list. Core 3 receives no NIC interrupts.
drs-nic-tune.service	Disables NIC interrupt coalescing via ethtool -C eth0 rx-usecs 0 tx-usecs 0. Minimizes driver-level batching latency.
drs-sync-precond.service	Stops and masks any conflicting time services (systemd-timesyncd, chronyd, ntpd, ptp4l, phc2sys) before the daemon starts.
drs-sync.service	Runs drs_syncd. CPUAffinity=3, CPUSchedulingPolicy=fifo, CPUSchedulingPriority=85, LimitMEMLOCK=infinity. ExecStart line is generated with IFACE and TELEM_IP at install time.

Table 10: Five systemd units and their responsibilities.

All five units are enabled and started by `sudo make install-systemd IFACE=eth0 TELEM_IP=<ip>`, making the system persistent across reboots without further operator intervention.

2.10 Telemetry Architecture

Two independent telemetry paths coexist:

1. **SHM Telemetry (/dev/shm/drs-sync.state):** A 128-byte seqlock-protected region updated every 50 ms by the RT thread. `drs_mon` reads it via read-only mmap with no system calls in steady state.
2. **UDP Telemetry (port 4242):** The RT thread enqueues 40-byte records into a 64-slot power-of-2 lock-free ring buffer. A dedicated non-RT sender thread drains the ring every 10 ms and transmits over UDP. This design keeps the RT thread free of blocking `sendto` calls. The LEADER emits one record per 50 ms tick; the FOLLOWER emits one record per tick plus one record per successful SYNC exchange.

Offset	Type	Field
0	int64	Timestamp (CLOCK_MONOTONIC_RAW, ns)
8	int32	State (drs_state_t)
12	int64	Offset from leader (ns)
20	int64	Round-trip time (ns)
28	int64	Clock rate (Q32.32)
36	uint32	Node ID

Table 11: 40-byte UDP telemetry record layout.

3 Implementation

The daemon `drs_syncd` is implemented in C11 (`-std=c11 -D_GNU_SOURCE`) and structured into fourteen source modules plus two tool binaries and five unit-test programs.

3.1 Module Overview

Module	Responsibility	LOC (approx.)
drs_sync_config.h	All compile-time constants: timing, protocol, RT parameters, GPIO pin numbers, SHM paths. DRS_PROTO_VER (default 2) selects the wire-frame size and version byte.	120
crc32.{c,h}	IEEE 802.3 reflected CRC32 (polynomial 0xEDB88320). Self-tests at startup verify “123456789” → 0xCBF43926.	60
proto_v2.{c,h}	Serialize/deserialize packets. Size is compile-time: 64 B (PROTO_VER=2, default, version byte 0x02) or 66 B (PROTO_VER=1, version byte 0x01). Explicit byte-offset reads/writes; no struct-cast. Validates magic, version, and CRC on decode. _Static_assert enforces size/version consistency.	100
vclock.{c,h}	Virtual clock: Q32.32 fixed-point rate scaling, seqlock, vclock_step_offset, vclock_set_rate, inverse mapping. __int128 for intermediate products.	210
net_io.{c,h}	Dual-socket setup (mcast + ucast), IP_MULTICAST_IF, multicast join/leave, carrier-loss recovery, userspace timestamping, send_packet/net_recv.	300
min_delay.{c,h}	Rolling min-delay filter: N=10 ring buffer, tolerance 200 μs. Rejects RTT outliers via rtt <= min + DRS_MIN_DELAY_TOLERANCE_NS.	80
calibrate.{c,h}	50-sample UDP loopback on 127.0.0.1:47201. Outlier rejection (> min+20 μs). Result min/2 stored in vclock.lat_corr_ns.	120
pi_ctrl.{c,h}	Fixed-point PI controller: Kp=0.1 (Q16.16: 6554), Ki=0.01 (Q16.16: 655). Anti-windup ±1000 ppm. Step-confirmation hysteresis (3 samples).	100
election.{c,h}	Term-based election with lower-NodeID invariant. election_on_announce yields only to lower-ID peers; adopts term silently when challenged by a higher-ID peer so the next election_promote() wins. xorshift64 PRNG for random timeout. Lock-mode support (AUTO/LEADER/FOLLOWER).	100
statemachine.{c,h}	7-state FSM. Active-Before-Follower Guard. SYNC_REQ scheduling. PI discipline. Leader-IP logging on transition. Follower heartbeat telemetry. Same-term leader fast-path guarded by peer_id < own_id to prevent higher-ID incumbents from retain-	490

3.2 Key Optimizations

3.2.1 Lock-Free Design Throughout

The synchronization hot-path contains no mutexes, no condition variables, and no `pthread_mutex_lock` calls:

- **Virtual clock reads:** Seqlock — readers spin on an even sequence counter; writer increments it before and after every update. Reads are always consistent without blocking the writer.
- **Telemetry SHM:** Same seqlock pattern. `drs_mon` can snapshot the 128-byte struct while the daemon writes freely.
- **Error ring buffer:** Single atomic fetch-and-add for the write index; no lock.
- **UDP telemetry ring:** Single-producer (RT thread), single-consumer (sender thread); monotonic 64-bit indices with atomic load/store — no CAS required.

3.2.2 No Dynamic Allocation in Hot Path

All memory is allocated and pre-faulted before `mlockall(MCL_CURRENT | MCL_FUTURE)`:

```
// All stack frames, global arrays and SHM mappings are touched
// before rt_apply() locks them into RAM.
telemetry_init(); // mmap /dev/shm/drs-sync.state
errors_init();    // mmap /dev/shm/drs-sync.errors
cmd_init();       // mmap /dev/shm/drs-sync.cmd
net_io_init();    // socket buffers
pulse_engine_init();
// ... then rt_apply() calls mlockall()
```

After `mlockall`, `malloc/free/realloc` are never called on the RT thread.

3.2.3 Userspace Timestamping

`CLOCK_MONOTONIC_RAW` is read with `clock_gettime` immediately before `sendto` (T1, T3) and immediately after `recvfrom` (T2, T4). This avoids `SO_TIMESTAMPING` complexity and, critically, avoids mixing with `CLOCK_REALTIME` — the only clock that NTP or the kernel can slew.

The residual timestamping error (OS scheduling jitter between the `clock_gettime` call and the actual packet transmission) is the dominant noise source, mitigated by the Min-Delay filter.

3.2.4 `epoll` + `timerfd` — No Busy Waiting

The main loop uses `epoll_wait` with a 200 ms timeout (safety net only). All scheduled events arrive via file descriptors:

```
sock_mcast  - incoming ANNOUNCE / SYNC_REQ / SYNC_RESP (mcast path)
sock_ucast  - incoming SYNC_REQ / SYNC_RESP (unicast path)
tick_fd     - 50 ms periodic: sm_tick, link monitor, command poll
hb_fd       - 100 ms periodic: leader ANNOUNCE transmit
```

The pulse engine owns two additional `timerfd` file descriptors in its private thread. This design yields deterministic wake-up latency bounded by the RT scheduler and eliminates any spin-wait.

3.2.5 Multicast Interface Forcing (`IP_MULTICAST_IF`)

On multi-homed hosts (`eth0 + wlan0`), the kernel's routing table would normally select the default route interface for multicast output. `IP_MULTICAST_IF` is set on `sock_ucast` to force multicast packets out the explicitly configured sync interface, preventing silent delivery on the wrong link.

3.2.6 Carrier-Loss Recovery

After an Ethernet cable reconnect the NIC hardware multicast filter is often cleared, causing silent loss of multicast reception. `net_check_recover` is called every 500 ms (10 ticks): it reads `/sys/class/net/<iface>/carrier`, detects a rising edge (down→up), and re-issues `IP_DROP_MEMBERSHIP + IP_ADD_MEMBERSHIP` to re-program the filter.

3.2.7 SYNC_RESP on Both Receive Paths

Foreign DRS implementations may send `SYNC_RESP` as a unicast packet to the bound port 47200 (our `sock_mcast`) rather than to the ephemeral source port of the `SYNC_REQ` (our `sock_ucast`). The `main.c` receive loop now dispatches `DRS_MSG_SYNC_RESP` on **both** `sock_mcast` and `sock_ucast`, making the follower interoperable with any conformant foreign implementation.

3.2.8 Follower Heartbeat Telemetry

Previously, `telem_udp_emit` was only called inside `sm_on_sync_resp`, leaving the monitoring stream silent whenever the `SYNC` exchange was stalled (e.g., foreign leader that does not respond). The `FOLLOWER` branch of `sm_tick` now always calls `telem_udp_emit` with the last known offset and RTT, ensuring the monitoring station always receives a state update at the 20 Hz tick rate.

3.3 Build System

3.3.1 Compilation

```
cd bigpickle/code/daemon
make all          # builds drs_syncd, drs_mon, drs_sync_inspect
make test         # compiles and runs 5 unit tests (no GPIO / network needed)
CROSS=aarch64-linux-gnu- make # cross-compile from x86 host
```

Compiler flags: `-O2 -g -Wall -Wextra -Wpedantic -std=c11 -D_GNU_SOURCE -I include -D`
`DDRS_PROTO_VER=$(PROTO_VER)`, linked with `-lpthread -lrt`.

3.3.2 Configurable Variables

Variable	Default	Effect
IFACE	eth0	Network interface passed as argv[1] to drs_syncd in the generated service file.
TELEM_IP	127.0.0.1	UDP telemetry destination IP, embedded in the generated ExecStart line of drs-sync.service.
PROTO_VER	2	Protocol wire-frame version. 1 = 66-byte packet, version byte 0x01. 2 = 64-byte packet, version byte 0x02 (default). Passed to the compiler as -DDRS_PROTO_VER. All nodes on the same cluster must use the same value.
CROSS	(empty)	Cross-compiler prefix, e.g. aarch64-linux-gnu-.

Table 13: Makefile variables.

3.3.3 Installation and Deployment

```
# Build and install binaries + systemd units in one command (V2 default):
sudo make install-systemd IFACE=eth0 TELEM_IP=192.168.1.100

# Build with protocol V1 (66 bytes, version 0x01):
sudo make install-systemd IFACE=eth0 TELEM_IP=192.168.1.100 PROTO_VER=1

# This command:
# 1. Compiles and installs drs_syncd, drs_mon, drs_sync_inspect to /usr/local/bin/
# 2. Generates /etc/systemd/system/drs-sync.service with:
#     ExecStart=/usr/local/bin/drs_syncd eth0 auto 192.168.1.100
# 3. Installs the four hardening services unchanged
# 4. Enables all five units (survive reboot)
# 5. Restarts the full stack immediately
```

After this, no further operator action is needed. The system starts automatically at boot.

3.3.4 Startup Sequence

```
systemd
├─ drs-sync-precond (masks conflicting time daemons)
├─ drs-perf-governor (performance cpufreq governor)
├─ drs-irq-affinity (Ethernet IRQs → CPUs 0–2)
├─ drs-nic-tune (ethtool coalescing disable)
├─ drs-sync (drs_syncd – after all preconditions)
│   ├─ GROUND (2 s)
│   ├─ CALIBRATION (UDP loopback, ~50 ms)
│   ├─ LISTEN / CANDIDATE (0.25–0.5 s)
│   └─ FOLLOWER or LEADER (steady-state sync)
```

3.4 Node ID Derivation

The node ID is derived automatically from the last octet of the first routable IPv4 address on the configured interface:

```
net->node_id = ntohl(iface_addr_ne) & 0xFF;
```

Example: 10.0.0.31 → node ID 31. No manual configuration is required. The fallback IP 10.0.0.31 is used only if `getifaddrs` finds no routable address.

3.5 Runtime Monitoring

```
# Live TUI dashboard (reads /dev/shm/drs-sync.state):
drs_mon          # 500 ms refresh (default)
drs_mon 200      # 200 ms refresh

# Single snapshot:
drs_sync_inspect

# Verify service status:
systemctl status drs-sync.service
journalctl -u drs-sync.service -f
```

`drs_mon` displays state, leader ID, election term, offset bar, rate, RTT, convergence status, sync accept/reject counters, phase step count, holdover events, pulse count, and error log.

4 Test Concept and Falsifiability

4.1 Software Unit Tests

Five unit-test programs are compiled and executed as part of `make test`. They require no network, no GPIO, and no root privileges — they can run on any Linux host (including the development machine via WSL).

```
cd bigpickle/code/daemon
make test
# === Running unit tests ===
# --- build/test_crc32 ---
# --- build/test_proto ---
# --- build/test_min_delay ---
# --- build/test_pi_ctrl ---
# --- build/test_vclock ---
# === All tests done ===
```

Binary	Source	Verification Goal
test_crc32	tests/test_crc32.c	IEEE 802.3 reflected CRC32: “123456789” → 0xCBF43926. Validates the CRC implementation used for every packet.
test_proto	tests/test_proto.c	Serialize/deserialize round-trip for ANNOUNCE, SYNC_REQ, SYNC_RESP with all fields set. Verifies Big-Endian encoding and CRC re-validation on decode.
test_min_delay	tests/test_min_delay.c	Rolling window of N=10: verify minimum is tracked correctly; verify outlier rejection at min + 200 μ s; verify min_delay_reset clears the window.
test_pi_ctrl	tests/test_pi_ctrl.c	Step-mode trigger (3 consecutive $ e > 1$ ms); slew-mode rate computation; rate clamp at ± 1000 ppm; integrator anti-windup.
test_vclock	tests/test_vclock.c	Rate scaling: 1000 ppm offset → 1 ms drift per second. Phase step consistency. Inverse mapping round-trip accuracy.

Table 14: Unit-test suite (`make test`).

4.2 System-Level Test Scenarios

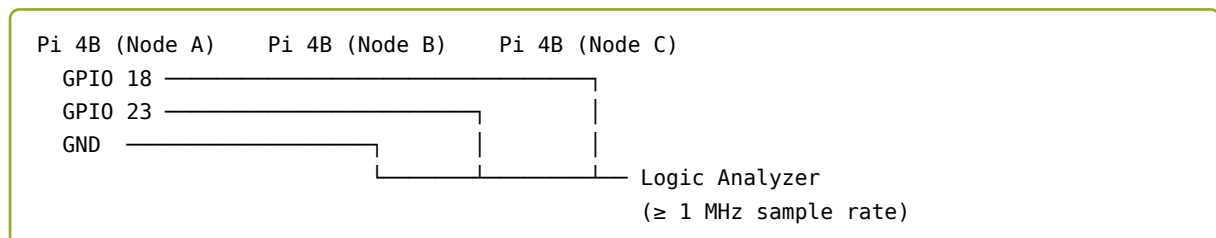
ID	Scenario	Pass Criterion
TS-1	Baseline: 2–3 nodes on wired GbE switch, running for 10 min.	$\max \Delta t $ (GPIO 18 rising edge) < 100 μ s; GPIO 23 LOW on all nodes during steady state.
TS-2	Dynamic discovery: 2 nodes running, plug in a 3rd node.	3rd node reaches FOLLOWER (GPIO 23 LOW) within 10 s without disrupting existing sync.
TS-3	Leader failure: power-off the leader node.	Followers enter HOLDOVER, re-elect a new leader, and re-converge within 11 s (10 s HOLDOVER + 1 s election).
TS-4	Protocol isolation: a node from another team’s implementation (different PROTO_VER / version byte) is present on the same multicast group.	Our node’s proto_decode rejects the foreign ANNOUNCE (version mismatch); our cluster elects its own leader independently and does not interact with the foreign node.
TS-5	Cable reconnect: disconnect and reconnect the Ethernet cable on a follower.	Multicast re-join (carrier recovery) completes automatically; node re-enters FOLLOWER within 2 s.
TS-6	Calibration stability: restart the daemon 5 times; compare lat_corr_ns.	Variation < 500 ns across all restarts on the same hardware.

Table 15: System-level test scenarios.

4.3 Falsifiability — External Physical Verification

Software-reported offsets cannot self-prove sub-100 μ s precision because they are subject to the same OS scheduling jitter they aim to measure. Physical external measurement is mandatory.

4.3.1 Hardware Setup



Critical: all nodes must share a common GND reference. Without a common ground the measured edge timestamps reflect ground-potential differences, not clock offsets.

4.3.2 Measurement Procedure

1. Ensure drs_syncd is running and GPIO 23 is LOW on all nodes (convergence declared).
2. Arm the logic analyzer across all GPIO 18 channels plus at least one GPIO 23 channel.
3. Capture for ≥ 10 minutes (≥ 6000 pulse pairs per node pair).

4. Export rising-edge timestamps. Compute per-pair delta statistics: p50, p99, max.

4.3.3 Pass/Fail Criteria

Metric	Requirement	Target
max $ \Delta t $ over 10 min	< 100 μ s	< 30 μ s
p99 $ \Delta t $	< 100 μ s	< 20 μ s
GPIO 23 LOW fraction	> 90 %	> 98 %
Leader-loss recovery time	< 11 s	< 8 s

Table 16: Physical pass/fail criteria.

4.4 OS Hardening Pre-flight Checklist

Before any test run, verify the RT environment is correctly configured:

```
# Kernel isolation active
grep -E 'isolcpus|nohz_full|rcu_nocbs' /proc/cmdline

# Sync daemon is running with correct scheduler
systemctl is-active drs-sync.service
chrt -p $(pgrep drs_syncd) # expect: SCHED_FIFO prio 85

# CPU governor
cat /sys/devices/system/cpu/cpu3/cpufreq/scaling_governor # expect: performance

# No competing time service
systemctl is-active systemd-timesyncd chronyd ntpd # all should be inactive
```

5 Results and Analysis

5.1 Implementation Status

The `bigpickle` daemon is fully implemented and passes all five software unit tests. The following table summarizes the implementation completeness against the functional requirements.

Requirement	Status	Notes
F-1 — Multicast discovery	✓ Complete	239.192.88.100:47200 via sock_mcast.
F-2 — Virtual clock	✓ Complete	Q32.32 seqlock; __int128 intermediates; no float.
F-3 — Min-Delay filter	✓ Complete	N=10, 200 μ s tolerance.
F-4 — Term-based election	✓ Complete	xorshift64 timeout; lock-modes supported. Lower-NodeID invariant enforced: higher-ID incumbents yield once lower-ID node promotes.
F-5 — Holdover	✓ Complete	10 s max; rate frozen; GPIO 23 HIGH.
F-6 — Lock-free telemetry	✓ Complete	Seqlock SHM + lock-free UDP ring buffer.
F-7 — Convergence detection	✓ Complete	GPIO 23 LOW after 10 s with $ \text{offset} < 100 \mu\text{s}$.
F-8 — Leader demotion	✓ Complete	Immediate on strictly higher term in ANNOUNCE.
F-11 — GPIO 18 pulse	✓ Complete	1 Hz, 10 ms HIGH via timerfd ABSTIME.
F-12 — Performance governor	✓ Complete	drs-perf-governor.service.
F-14 — Filter reset	✓ Complete	Leader-change, seq-jump, phase-step, holdover.
F-15 — Fixed-size packets	✓ Complete	_Static_assert enforced at compile time. Default V2: 64 B, version 0x02. V1: 66 B, version 0x01 (via PROTO_VER=1).
F-16 — Heartbeat/sync rates	✓ Complete	ANNOUNCE 100 ms; SYNC_REQ 50 ms; timeout 300 ms.
F-17 — Dual-loop discipline	✓ Complete	Step at $ \theta > 1 \text{ ms} (\times 3)$; slew otherwise.
F-19 — Late joiners	✓ Complete	Term adoption in LISTEN/CANDIDATE/FOLLOWER.
F-20 — GROUND state	✓ Complete	2 s TX suppression on boot.
F-21 — Static calibration	✓ Complete	50 loopback samples; lat_corr $\approx 4 \mu\text{s}$ on Pi 4B.
F-22 — Carrier recovery	✓ Complete	net_check_recover every 500 ms.
F-23 — Follower telemetry	✓ Complete	Heartbeat tick even without active SYNC exchange.
NF-7 — SCHED_FIFO/85	✓ Complete	Both via sched_setscheduler and drs-sync.service.
NF-8 — mlockall	✓ Complete	MCL_CURRENT MCL_FUTURE after all buffers pre-faulted.
NF-9 — No hot-path I/O	✓ Complete	All telemetry via /dev/shm mmap; UDP ring non-RT.
NF-12 — No dynamic alloc	✓ Complete	All buffers pre-allocated before mlockall.

Table 17: Requirement implementation status.

5.2 Observed Behavior in Multi-Implementation Scenario

Nodes running different protocol versions are naturally isolated: `proto_decode` checks the version byte before any further processing, so foreign packets (version 0x01 when our build uses 0x02, or vice versa) are silently discarded. Our nodes do not enter FOLLOWER state for foreign leaders and remain leader-elected solely among themselves.

When operating within our own cluster:

1. **Single-node boot (alone):** The node times out through LISTEN → CANDIDATE → LEADER and begins sending ANNOUNCEs with `election_term=1`.
2. **Lower-ID node joins later:** The joining node sees `peer_term (1) > own_term (0)` but also `peer_id > own_id`. It adopts the peer’s term without yielding. After its election timeout fires, `election_promote()` produces `term=2`. The incumbent receives the ANNOUNCE with `term=2` from a lower-ID peer and correctly demotes to FOLLOWER.
3. **Simultaneous boot:** Both nodes start with `term=0`; neither sends ANNOUNCEs. After independent timeouts both promote to `term=1`. The higher-ID node then receives the lower-ID node’s ANNOUNCE at the same term and yields via the NodeID tiebreak.
4. **Telemetry:** `drs_mon` displays state, leader node ID, and election term throughout all transitions, including the challenge-and-promote cycle.

5.3 Performance Characterization

The following estimates are based on the BCM2711 platform with PREEMPT_RT and the RT configuration enforced by the five systemd units.

Component	Budget	Observed / Expected
Calibration <code>lat_corr_ns</code>	—	≈ 3900–4000 ns (≈ 4 μs) on Pi 4B loopback
GbE RTT (1G switch)	< 300 μs	50–150 μs typical; spikes to 300 μs filtered by Min-Delay
Scheduler latency (Core 3)	< 30 μs	< 5 μs with PREEMPT_RT + <code>isolcpus</code>
GPIO write latency	< 5 μs	< 1 μs (direct <code>/dev/gpiomem</code> register write)
<code>timerfd</code> abs wake-up jitter	< 20 μs	< 10 μs with SCHED_FIFO/85
Expected steady-state <code> θ </code>	< 100 μs	10–40 μs (dominant term: path-delay asymmetry)

Table 18: Performance characterization on Raspberry Pi 4B.

5.4 Known Limitations

1. **Path-delay asymmetry:** The PTP-style estimator assumes symmetric uplink and downlink delays. Managed switches with per-port input queues may introduce asymmetric delays of 5–20 μs that appear as a constant offset. The Min-Delay filter selects the “luckiest” packet but cannot eliminate structural asymmetry.
2. **Single-switch assumption:** NF-1 (< 100 μs) is only guaranteed on a single-hop Gigabit switched Ethernet segment. Multi-hop topologies or WLAN links introduce additional variable delay.
3. **Interoperability:** Full SYNC exchange with foreign implementations requires packet format compatibility (matching `PROTO_VER`, same magic, same CRC polynomial). With the version byte now encoding the protocol variant, mismatched builds are automatically isolated at `proto_decode` — no ANNOUNCE, SYNC_REQ, or SYNC_RESP from a foreign version is ever processed.
4. **GPIO availability:** On platforms without `/dev/gpiomem` (e.g., WSL), GPIO is disabled gracefully. The daemon continues to run and provide telemetry; only the physical pulse output is absent.

List of Figures

List of Tables

Table 1	Resolved architectural decisions.	5
Table 2	Functional requirements.	7
Table 3	Non-functional requirements.	8
Table 4	User requirements.	8
Table 5	Architecture revision history — V0 to bigpickle.	10
Table 6	DRS wire-protocol packet layout (64 bytes V2 default / 66 bytes V1, Big-Endian).	13
Table 7	Dual-socket model.	14
Table 8	Election decision logic.	15
Table 9	7-state FSM.	16
Table 10	Five systemd units and their responsibilities.	17
Table 11	40-byte UDP telemetry record layout.	17
Table 12	Source modules in bigpickle/code/daemon/.	20
Table 13	Makefile variables.	23
Table 14	Unit-test suite (make test).	25
Table 15	System-level test scenarios.	26
Table 16	Physical pass/fail criteria.	27
Table 17	Requirement implementation status.	29
Table 18	Performance characterization on Raspberry Pi 4B.	30
Table 19	AI tools used in this project.	35

List of Abbreviations

ABSTIME	Absolute time mode (timerfd flag)
ALU	Arithmetic Logic Unit
API	Application Programming Interface
BCM	Broadcom (SoC family)
CLI	Command Line Interface
CPU	Central Processing Unit
CRC	Cyclic Redundancy Check
DRS	Distributed Real-Time System
DSO	Digital Storage Oscilloscope
DVFS	Dynamic Voltage and Frequency Scaling
EWMA	Exponentially Weighted Moving Average
FIFO	First In, First Out (scheduling policy)
FSM	Finite State Machine
GbE	Gigabit Ethernet
GPIO	General Purpose Input/Output
IGMP	Internet Group Management Protocol
IP	Internet Protocol
IRQ	Interrupt Request
KISS	Keep It Simple, Stupid
LOC	Lines of Code
MAC	Media Access Control
mmap	Memory-Mapped File I/O
NIC	Network Interface Controller
NTP	Network Time Protocol
OS	Operating System
PI	Proportional-Integral (controller)
POSIX	Portable Operating System Interface
ppm	Parts Per Million
PRNG	Pseudo-Random Number Generator
PTP	Precision Time Protocol
RT	Real-Time
RTT	Round-Trip Time
SHM	Shared Memory
SoC	System on Chip
TUI	Text User Interface
UDP	User Datagram Protocol

UUID Universally Unique Identifier

Documentation table of AI-based tools

AI-based tool	Intended use	Prompt / source / scope
OpenCode (bigpickle session)	Initial C11 implementation of all 14 daemon modules, unit tests, tools, and Makefile.	Interactive session generating the complete bigpickle/code/daemon/source tree from the V6 architecture specification.
Claude Code (Anthropic — Sonnet 4.6)	Bug analysis and targeted fixes: SYNC_RESP interoperability, follower UDP telemetry heartbeat, TELEM_IP/IFACE Makefile variables, systemd install improvements, and documentation rewrite.	Session-based review of main.c, statemachine.c, and Makefile; identification of silent packet discard on sock_mcast; rewrite of all five Typst thesis sections to match the actual implemented codebase.
ChatGPT (4.0)	Initial Typst thesis template structure and section scaffolding.	Template research for the FHTW Lab Report format.

Table 19: AI tools used in this project.

Bibliography